

Despliegue Automatizado de un CRUD con Jenkins

Objetivo del Laboratorio:

Introducir los conceptos de integración continua y despliegue continuo (CI/CD) mediante la creación de un flujo básico de despliegue automatizado usando Jenkins. Los estudiantes aprenderán a:

1. Construir un archivo .jar desde su aplicación CRUD existente.
 2. Desplegar el archivo JAR manualmente.
 3. Configurar un trabajo en Jenkins para automatizar el despliegue.
-

Requisitos previos

- CRUD ya desarrollado en Java (o lenguaje compatible con JAR).
 - Java Development Kit (JDK) instalado.
 - Jenkins instalado localmente o en un contenedor Docker.
 - Conocimiento básico sobre comandos de terminal.
 - Git configurado para manejar el proyecto.
-

Materiales

1. Máquina con Java y Jenkins pre-instalados.
 2. Acceso a un repositorio Git (por ejemplo, GitHub).
 3. Navegador web para acceder a Jenkins.
-

Parte 1: Empaquetado del CRUD en un archivo JAR

1. **Abre la terminal** y navega al directorio raíz del proyecto CRUD.
2. Ejecuta el siguiente comando para compilar y empaquetar el proyecto:

```
mvn clean package
```

3. Verifica que se haya generado un archivo .jar en la carpeta target/ (por ejemplo, zoo-fantastic-app.jar).
4. **Ejecuta el JAR manualmente** para probar que funciona:

Principios de Desarrollo de software

```
java -jar target/zoo-fantastic-app.jar
```

Parte 2: Despliegue manual

1. Si la aplicación corre correctamente, accede a <http://localhost:8080> (o el puerto configurado) para validar que el CRUD esté en funcionamiento.
 2. **Detén la aplicación** presionando Ctrl + C.
-

Parte 3: Configuración de Jenkins

1. **Accede a Jenkins** desde tu navegador en <http://localhost:8080>. (tener cuidado con los puertos usados por la app, ambos estan en el 8080)
2. **Crea un nuevo trabajo (job)** siguiendo estos pasos:
 - Ve a “New Item” y selecciona **Freestyle Project**.
 - Nombra el proyecto ej: Despliegue-CRUD.
 - En la sección **Source Code Management**, selecciona Git e ingresa la URL del repositorio del proyecto.
3. En la sección **Build**, agrega un paso para ejecutar comandos de shell:

```
mvn clean package
```

4. En la sección **Post-build Actions**, añade un comando para desplegar el JAR:

```
java -jar target/crud-app.jar &
```

Parte 4: Prueba del flujo automatizado

1. **Guarda el trabajo en Jenkins** y haz clic en “Build Now” para probarlo.
2. **Verifica** que Jenkins descargue el código, genere el JAR y lo despliegue correctamente.
3. Accede a la aplicación en el navegador para confirmar que está en funcionamiento.

Principios de Desarrollo de software

Parte 5: Desplegando el CRUD usando Docker

Objetivo de esta sección

Mejorar la automatización y la portabilidad del CRUD empaquetándolo en un contenedor Docker y desplegándolo mediante Jenkins.

Requisitos adicionales

- **Docker** instalado (puede ser Docker Desktop o Docker Engine).
- Acceso a **Docker Hub** (opcional) para publicar la imagen.
- Docker Plugin activado en Jenkins (instrucciones disponibles en la documentación oficial de Jenkins).

Paso 1: Creación del Dockerfile

1. En la raíz del proyecto CRUD, crea un archivo llamado **Dockerfile** con el siguiente contenido:

```
dockerfile
Copy code
FROM openjdk:11-jre-slim
WORKDIR /app
COPY target/zoo-fantastic-app.jar zoo-fantastic-app.jar
EXPOSE 8080
CMD ["java", "-jar", "zoo-fantastic-app.jar"]
```

Explicación:

- Se utiliza la imagen base de OpenJDK (esto está para java 11, revisar según su versión de java si puede presentar inconvenientes).
 - Se copia el archivo .jar generado en el directorio /app del contenedor.
 - Se expone el puerto 8080.
 - Se configura el comando que ejecutará la aplicación.
2. **Construye y prueba la imagen Docker:**

```
docker build -t zoo-fantastic-app:latest .
```

```
docker run -p 8080:8080 zoo-fantastic -app:latest
```

Verifica que la aplicación esté corriendo en <http://localhost:8080>.

Paso 2: Configuración del Pipeline en Jenkins para Docker

1. **Accede a Jenkins** y crea un nuevo trabajo tipo **Pipeline**.
2. En la configuración del Pipeline, selecciona “Pipeline script” e ingresa el siguiente código:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package'
      }
    }
    stage('Docker Build') {
      steps {
        sh 'docker build -t crud-app:latest .'
      }
    }
    stage('Docker Run') {
      steps {
        sh 'docker run -d -p 8080:8080 crud-app:latest'
      }
    }
  }
}
```

¡Ojo! Analizar que hace este código, es muy sencillo, pero revisar lo que está haciendo

3. **Ejecuta el Pipeline** y verifica que Jenkins:
 - Compila el proyecto.
 - Construye la imagen Docker.
 - Despliega el contenedor automáticamente.
-

Principios de Desarrollo de software

Parte 6: Añadir Pruebas Unitarias al Pipeline

1. **Asegúrate de tener pruebas unitarias definidas** tus pruebas deben ubicarse en `src/test/java`.
2. **Modifica el archivo pom.xml** (para proyectos Maven) si no tienes la configuración básica para ejecutar las pruebas:

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>2.22.2</version>
      <configuration>
        <skipTests>>false</skipTests>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Explicación: El plugin maven-surefire-plugin se usa para ejecutar las pruebas con Maven.

Paso 7: Actualización del Pipeline de Jenkins

Aquí está el nuevo **script del pipeline** que incluye la ejecución de pruebas:

```
pipeline {
  agent any

  stages {
    stage('Build') {
      steps {
        sh 'mvn clean package -DskipTests=false'
      }
    }
  }
}
```

Principios de Desarrollo de software

```
stage('Run Tests') {
    steps {
        sh 'mvn test'
    }
}

stage('Docker Build') {
    steps {
        sh 'docker build -t zoo-fantastic-app:latest .'
    }
}

stage('Docker Run') {
    steps {
        sh 'docker run -d -p 8080:8080 zoo-fantastic-app:latest'
    }
}

post {
    always {
        echo 'Pipeline terminado.'
    }
    success {
        echo '¡Despliegue exitoso!'
    }
    failure {
        echo 'Error en el pipeline. Revisar los logs.'
    }
}
```

Descripción del Nuevo Pipeline

1. **Stage 'Build'**: Se compila el proyecto con Maven.
2. **Stage 'Run Tests'**: Se ejecutan las pruebas unitarias usando Maven.

Principios de Desarrollo de software

3. **Stage 'Docker Build'**: Si las pruebas pasan, se construye la imagen Docker.
 4. **Stage 'Docker Run'**: Despliega la aplicación dentro de un contenedor.
-

Validación del Pipeline

1. Guarda el trabajo en Jenkins y haz clic en "**Build Now**".
 2. Si alguna prueba falla, el pipeline se detendrá, y no se ejecutará el despliegue.
 3. Verifica los **logs de Jenkins** para identificar problemas en caso de errores.
-

Recursos útiles sobre Jenkins

1. **Documentación oficial de Jenkins:**
<https://www.jenkins.io/doc/>
(Guía completa sobre instalación, configuración y uso de Jenkins).
2. **Guía de Pipeline de Jenkins:**
<https://www.jenkins.io/doc/book/pipeline/>
(Explicación sobre los tipos de pipelines y cómo escribirlos).
3. **Integración de Docker en Jenkins:**
<https://plugins.jenkins.io/docker-plugin/>
(Documentación del plugin de Docker para Jenkins).
4. **Jenkins en contenedores:**
<https://www.jenkins.io/doc/book/installing/docker/>
(Guía para ejecutar Jenkins dentro de un contenedor Docker).

Tips para el Informe:

Principios de Desarrollo de software

1. **Experiencia con Docker:** Incluir el contenido del Dockerfile y capturas de la imagen funcionando.
2. **Pipeline de Jenkins:** Explicar cada etapa del Pipeline y los beneficios de usar contenedores.
3. **Desafíos encontrados:** Problemas con permisos o configuración de Docker y cómo los resolvieron.